

# 程式碼實驗室：Spanner MCP 伺服器

來源：<https://codelabs.developers.google.com/spanner-mcp-server?hl=zh-tw>

步驟數：5

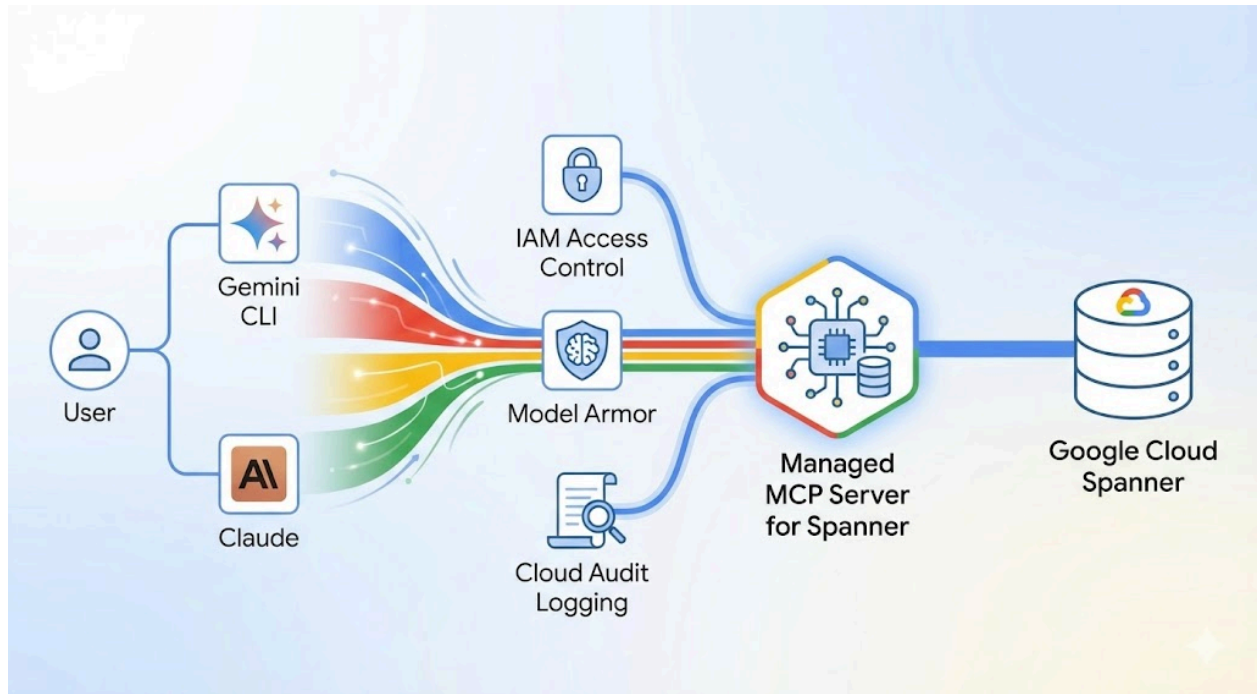
---

# 目錄

1. [總覽](#)
2. [設定 Spanner 多模型環境](#)
3. [設定 Gemini CLI 和 Spanner MCP](#)
4. [生成多模態模型，包括以圖表為主的查詢](#)
5. [恭喜！](#)

## 1. 總覽

本程式碼實驗室會示範如何使用 `gemini-cli` 和 Google 的 **Model Context Protocol (MCP)** 伺服器，在 **Spanner** 上發出圖形查詢。將 **Spanner** 的多模型資料庫 (包括 **Spanner Graph**) 與代管 MCP 服務整合後，您就能在單一資料庫中，以自然語言找出資料中隱藏的關係 (例如詐欺集團或供應鏈依附關係)，不必手動編寫複雜的 **Graph Query Language** (圖形查詢語言) 或 **SQL** 查詢。



### 學習目標

- 瞭解如何設定 **Spanner** 多模型資料庫，包括 **MCP** 適用的 **Spanner Graph**。
- 如何將 **Gemini CLI** 連線至代管的遠端 **Spanner** 服務端點。
- 瞭解如何使用自然語言生成 **SQL**，並自動整合 **GQL**，執行圖形遍歷和模式比對，以及關聯式作業。
- 保護 **AI** 工作流程中圖形資料的最佳做法。

### 必要條件

- 已啟用計費功能的 **Google Cloud** 專案。
- 存取終端機 (例如 **Cloud Shell**)。請參閱[這個](#)程式碼研究室。
- 已安裝並驗證 `gcloud` CLI。
- 存取 [Gemini CLI](#)。

注意：您將在 **Unix** 終端機執行所有指令。請參閱上方的必要條件，取得可存取 `gcloud` 的終端機，並完成適當的驗證。

## 2. 設定 Spanner 多模型環境

如果資料庫結構定義已預先定義，建議使用代管 MCP。我們會根據 Spanner Graph 範例，建立金融「詐欺偵測」圖表。

### 啟用 API



```
export PROJECT_ID="YOUR_PROJECT_ID"
```



```
gcloud config set project ${PROJECT_ID}

# Enable Spanner's MCP endpoint
gcloud beta services mcp enable spanner.googleapis.com --project=${PROJECT_ID}
```

### 建立資料庫結構定義

建立執行個體和資料庫，其中包含資料庫表格和屬性圖。這個結構會追蹤帳戶、人員，以及兩者之間的轉移。

注意：您可以在 Spanner Studio 中執行這些 DDL 陳述式。如要瞭解如何透過 GCP 主控台建立及查詢 Spanner 資料庫，請參閱[這個連結](#)。



```

CREATE TABLE Account (
    id INT64 NOT NULL,
    create_time TIMESTAMP,
    is_blocked BOOL,
    nick_name STRING(MAX),
) PRIMARY KEY(id);

CREATE TABLE Person (
    id INT64 NOT NULL,
    name STRING(MAX),
    birthday TIMESTAMP,
    country STRING(MAX),
    city STRING(MAX),
) PRIMARY KEY(id);

CREATE TABLE AccountTransferAccount (
    id INT64 NOT NULL,
    to_id INT64 NOT NULL,
    amount FLOAT64,
    create_time TIMESTAMP NOT NULL,
    order_number STRING(MAX),
    FOREIGN KEY(to_id) REFERENCES Account(id),
) PRIMARY KEY(id, to_id, create_time),
    INTERLEAVE IN PARENT Account ON DELETE CASCADE;

CREATE TABLE PersonOwnAccount (
    id INT64 NOT NULL,
    account_id INT64 NOT NULL,
    create_time TIMESTAMP,
    FOREIGN KEY(account_id) REFERENCES Account(id),
) PRIMARY KEY(id, account_id),
    INTERLEAVE IN PARENT Person ON DELETE CASCADE;

CREATE PROPERTY GRAPH FinGraph
    NODE TABLES(
        Account
            KEY(id)
            LABEL Account PROPERTIES(
                create_time,
                id,
                is_blocked,
                nick_name),

        Person
            KEY(id)
            LABEL Person PROPERTIES(
                birthday,
                city,
                country,
                id,
                name)
    )

```

```

EDGE TABLES(
  AccountTransferAccount
    KEY(id, to_id, create_time)
    SOURCE KEY(id) REFERENCES Account(id)
    DESTINATION KEY(to_id) REFERENCES Account(id)
    LABEL Transfers PROPERTIES(
      amount,
      create_time,
      id,
      order_number,
      to_id),

  PersonOwnAccount
    KEY(id, account_id)
    SOURCE KEY(id) REFERENCES Person(id)
    DESTINATION KEY(account_id) REFERENCES Account(id)
    LABEL Owns PROPERTIES(
      account_id,
      create_time,
      id)
);

```

## 填入資料

### 插入人員

```

INSERT INTO Person (id, name, birthday, country, city) VALUES
(1, 'Alex', '1990-01-01T00:00:00Z', 'USA', 'New York'),
(2, 'Lee', '1985-05-15T00:00:00Z', 'USA', 'San Francisco'),
(3, 'Dana', '1995-08-20T00:00:00Z', 'Canada', 'Vancouver'),
(4, 'Maria', '1992-04-15T00:00:00Z', 'USA', 'Chicago'),
(5, 'David', '1988-11-20T00:00:00Z', 'Canada', 'Toronto');

```

### 插入帳戶

```

INSERT INTO Account (id, create_time, is_blocked, nick_name) VALUES
(7, '2020-01-10T14:22:20.222Z', false, 'Vacation Fund'),
(16, '2020-01-28T01:55:09.206Z', true, 'Vacation Fund'),
(20, '2020-02-18T13:44:20.655Z', false, 'Rainy Day Fund'),
(25, '2023-01-15T10:00:00Z', false, 'Savings'),
(30, '2023-02-20T11:00:00Z', false, 'Checking');

```

### 將人員連結至帳戶

```
INSERT INTO PersonOwnAccount (id, account_id, create_time) VALUES
(1, 7, '2020-01-10T14:22:20.222Z'),
(2, 16, '2020-01-28T01:55:09.206Z'),
(3, 20, '2020-02-18T13:44:20.655Z'),
(4, 25, '2023-01-15T10:00:00Z'),
(5, 30, '2023-02-20T11:00:00Z');
```

## 新增轉移

```
INSERT INTO AccountTransferAccount (id, to_id, amount, create_time, order_number) VALUES
(7, 16, 300, '2024-02-28T10:00:00Z', 'ORD123'),
(7, 16, 100, '2024-02-29T11:00:00Z', 'ORD124'),
(16, 20, 300, '2024-02-29T12:00:00Z', 'ORD125'),
(20, 7, 500, '2024-02-29T13:00:00Z', 'ORD126'),
(20, 16, 200, '2024-02-29T14:00:00Z', 'ORD127'),
(7, 25, 150.75, '2024-03-01T12:00:00Z', 'ORD456'),
(25, 30, 200.00, '2024-03-02T14:30:00Z', 'ORD457'),
(30, 16, 50.25, '2024-03-03T16:00:00Z', 'ORD458');
```

---

步驟 3 · 約 0 分鐘

### 3. 設定 Gemini CLI 和 Spanner MCP

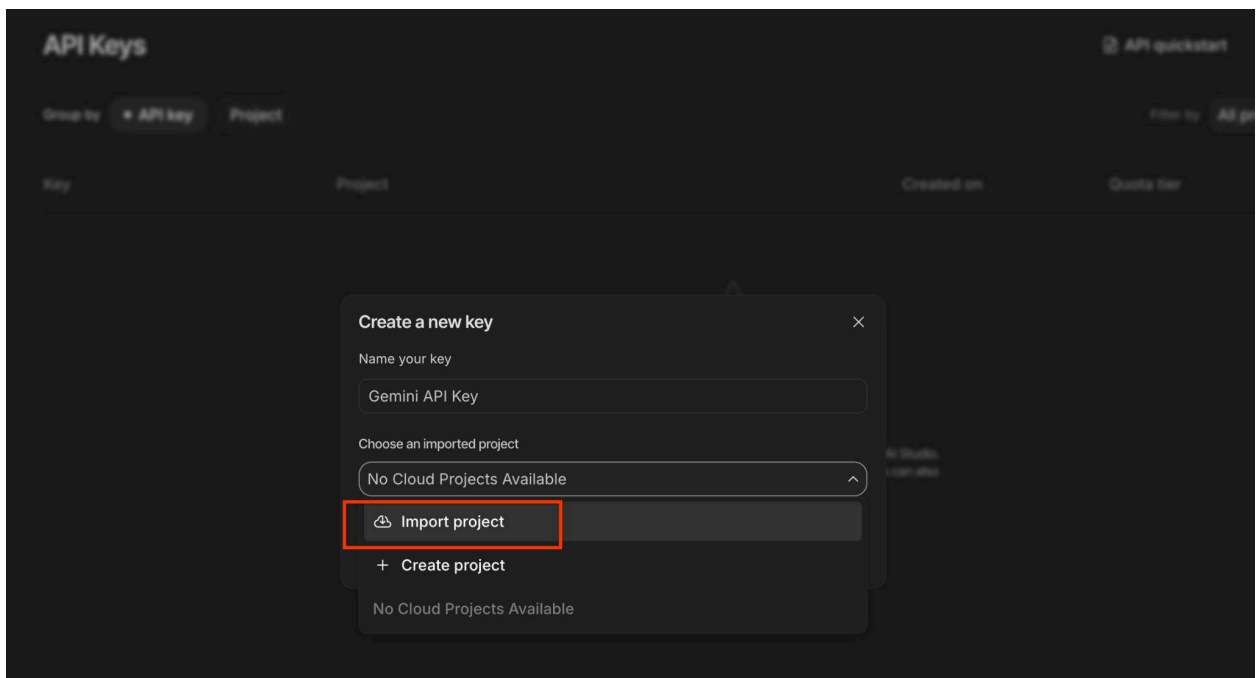
本實驗室的目標是使用 Gemini CLI 做為主要代理程式，執行大部分的動作，第一步是設定 Gemini CLI，使其能與遠端 Spanner MCP 伺服器搭配運作。此外，強烈建議使用最新版 Gemini CLI 和最新版 Gemini，例如 Gemini-3-pro 或 Gemini-3-flash (目前為搶先體驗版)。

如果您已安裝最新版 Gemini CLI，並設定使用最新版 Gemini 模型，可以略過接下來的 2 個步驟，直接前往步驟 3。

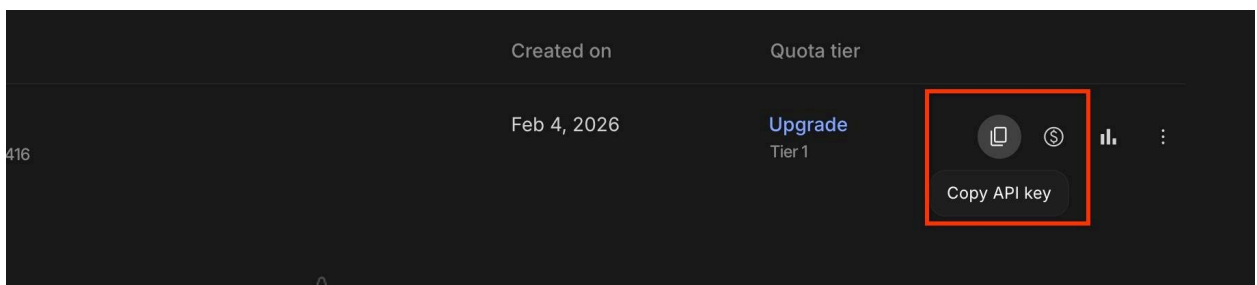
#### 取得 Gemini API 金鑰

如要在 Gemini CLI 中使用最新的 Gemini 模型，必須設定 API 金鑰驗證。我們可以在 Google AI Studio 中取得 API 金鑰

1. 在瀏覽器中前往 Google AI Studio -> 「取得 API 金鑰」。
2. 按一下畫面右側的 `Create API key` 按鈕
3. 在彈出式視窗中選擇專案，如果沒有看到專案，請匯入專案



4. 建立 API 金鑰。
5. 你可以使用右側的複製按鈕複製金鑰。



金鑰已準備就緒，現在可用於 Gemini CLI

#### 設定 Gemini CLI

您可以透過設定設定檔或使用 `/settings` 指令以互動方式設定 Gemini CLI。我們希望設定能保留在設定檔中。

首先，我們要為專案建立目錄。我們將目錄命名為 `projectspannermcp`，並使用下列指令切換至該目錄。

```
mkdir ~/projectspannermcp
cd ~/projectspannermcp
```

我們將在該目錄中建立 gemini 設定檔的目錄，並執行所有指令。根據預設，Gemini CLI 會使用使用者範圍設定檔，以及安裝在 ~/.gemini 目錄中的擴充功能。但我們會在 ~/projectspannermcp 目錄中建立自己的專案範圍設定。首先，我們來建立設定目錄和檔案。

```
mkdir ~/projectspannermcp/.gemini
touch ~/projectspannermcp/.gemini/settings.json
```

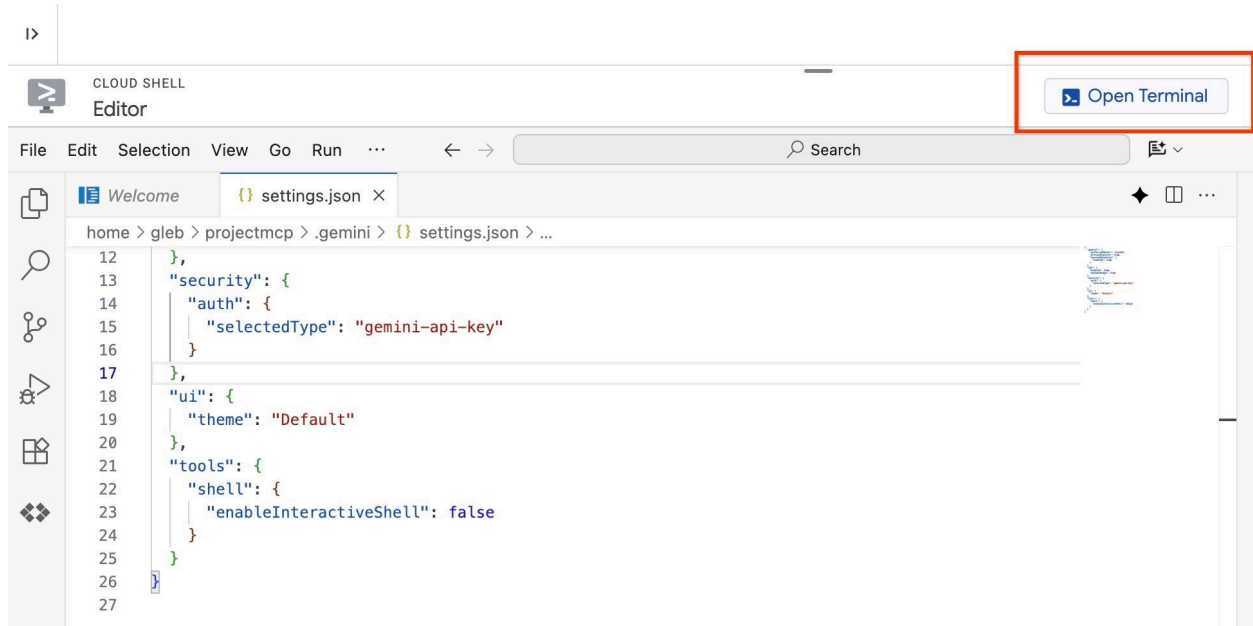
接著使用 Cloud Shell 編輯器開啟 ~/projectspannermcp/.gemini/settings.json 檔案。

```
edit ~/projectspannermcp/.gemini/settings.json
```

在開啟的編輯器視窗中，新增下列設定。

```
{
  "general": {
    "preferredEditor": "vscode",
    "previewFeatures": true,
    "sessionRetention": {
      "enabled": true
    }
  },
  "ide": {
    "enabled": true,
    "hasSeenNudge": true
  },
  "security": {
    "auth": {
      "selectedType": "gemini-api-key"
    }
  },
  "ui": {
    "theme": "Default"
  },
  "tools": {
    "shell": {
      "enableInteractiveShell": false
    }
  }
}
```

編輯檔案後，你可以使用 `ctrl+s` 儲存變更，然後按一下 `Open Terminal` 按鈕切換回終端機



上述 settings.json 檔案會將 Gemini CLI 驗證類型設為 gemini-api-key，您需要向模型提供金鑰。如要這麼做，請在要使用 Gemini CLI 的目錄中建立 .env 檔案，並將 Gemini CLI 驗證的 API 金鑰儲存為環境變數，IDE 啟動時就會擷取該變數。

使用 Cloud Shell 編輯器編輯檔案

```
edit ~/projectspannermcp/.env
```

並儲存您在上述「取得 Gemini API 金鑰」步驟中建立的 API 金鑰。

```
GEMINI_API_KEY= <YOUR_API_KEY>
```

現在可以返回終端機並繼續操作。下一步是將 Gemini CLI 更新至最新版本。Google Cloud Shell 或環境中的版本可能過時，無法正確搭配最新設定和 MCP 服務運作。因此請務必將 Gemini CLI 更新至最新版本並啟動。在終端機中執行下列指令。

```
cd ~/projectspannermcp/  
npm install -g @google/gemini-cli@latest
```

現在可以驗證版本。請注意，如果您在 Google Cloud Shell 中工作，每次重新啟動殼層時，系統可能會還原為舊版。您對 Gemini CLI 所做的變更不會永久保留在 Cloud Shell 中。

```
gemini --version
```

畫面應會顯示最新版本的 Gemini CLI (撰寫本實驗室時為 0.27 版)。如要確認最新版本的 Gemini CLI，請按[這裡](#)。現在我們可以啟動這項功能，並確認使用的是最新 Gemini 模型。在專案資料夾中執行下列指令，啟動 Gemini CLI。

```
gemini
```

啟動 Gemini CLI 後，您可以在介面中執行 `/about` 指令，查看版本和模型。您應該會看到類似下圖的畫面。

```
> /about

About Gemini CLI

CLI Version      0.27.0
Git Commit      ec10a7692
Model           auto-gemini-3
Sandbox         no sandbox
OS             linux
Auth Method     gemini-api-key
GCP Project     cloudshell-gca

2 GEMINI.md files | 1 MCP server

> █ Type your message or @path/to/file

~/projectmcp          no sandbox (see /docs)          Auto (Gemini 3) /model
```

你可以看到它使用的是最新的 Gemini-3 模型。以下步驟示範如何使用自然語言查詢，但需要使用 Gemini-3 模型。

## 設定 Gemini CLI 以使用 Spanner MCP

我們還要在 `settings.json` 檔案中新增一項內容。首先，請退出 Gemini CLI (你可以按兩次 Ctrl+C)。

在上述相同的 `settings.json` 中，我們要新增位元來設定 Spanner MCP 伺服器。下方是完整的 `settings.json`。請注意開頭為 `"mcpServers"` 的位元。

```
{
  "general": {
    "preferredEditor": "vscode",
    "previewFeatures": true,
    "sessionRetention": {
      "enabled": true
    }
  },
  "ide": {
    "enabled": true,
    "hasSeenNudge": true
  },
  "security": {
    "auth": {
      "selectedType": "gemini-api-key"
    }
  },
  "ui": {
    "theme": "Default"
  },
  "tools": {
    "shell": {
      "enableInteractiveShell": false
    }
  },
  "mcpServers": {
    "spanner": {
      "httpUrl": "https://spanner.googleapis.com/mcp",
      "authProviderType": "google_credentials",
      "oauth": {
        "scopes": [
          "https://www.googleapis.com/auth/spanner.admin",
          "https://www.googleapis.com/auth/spanner.data"
        ]
      },
      "timeout": 5000
    }
  },
  "mcp": {
    "allowed": [
      "spanner"
    ]
  }
}
```

請先驗證身分，再繼續執行下一個步驟

```
gcloud auth application-default login
```

完成驗證後，請再次啟動 gemini-cli

```
gemini
```



## 4. 生成多模態模型，包括以圖表為主的查詢

首先，請列出已設定專案中的 Spanner 執行個體。

「Please list all Spanner instances in my project <project1 > (replace with your project id)」(請列出專案<project1> 中的所有 Spanner 執行個體 (請替換為您的專案 ID))

回應：

```
The Spanner instances in your project are:
[instance1]
```

「Please list all databases in <instance1 > (replace with your instance)」。(請列出<instance1> 中的所有資料庫 (請替換為您的執行個體))

回應：

```
The Spanner databases in [instance1] are:
[database1]
```

接著，請告知 Gemini 執行 DDL 陳述式和插入陳述式的執行個體/資料庫，也就是 Spanner 設定部分：

「除非另有說明，否則所有查詢都會針對執行個體 <your-instance> 中的資料庫 <your-db> 提交」

**NOTE：**請使用您執行上述 DDL 陳述式和插入陳述式的執行個體/資料庫。

連線後，Gemini 模型就會瞭解圖形結構定義。現在您可以使用自然語言執行複雜的遍歷作業。

模型會自動將這些內容翻譯成 GQL (Graph Query Language)：

### 找出網路結構

「請說明『FinGraph』屬性圖的結構。節點和邊緣標籤為何？」

回應：

```
Node Labels
* `Account`: Represents financial accounts.
    * Properties: id, create_time, is_blocked, nick_name
* `Person`: Represents individuals.
    * Properties: id, name, birthday, country, city

Edge Labels
* `Transfers`: Represents a transfer between two Account nodes.
    * Source: Account
    * Destination: Account
    * Properties: id, to_id, amount, create_time, order_number
* `Owns`: Represents a Person owning an Account.
    * Source: Person
    * Destination: Account
    * Properties: id, account_id, create_time
```

# 偵測詐欺和循環

「找出所有循環轉帳，也就是從帳戶 A 匯款，並在 3 次轉移內最終返回帳戶 A 的交易。」

回應：

The analysis confirms the following circular transfers where money returns to the originating account within 3 hops:

3-Hop Cycles

Money travels through two intermediate accounts before returning to the origin.

- \* Account 16 (Involved in multiple 3-hop paths)
- \* Account 20 (Involved in multiple 3-hop paths)
- \* Account 7 (Involved in multiple 3-hop paths)

These accounts are participating in circular flow patterns of varying lengths.

# 深入關係分析

「找出所有帳戶擁有者，這些帳戶在過去 24 個月內，曾將超過 \$499 美元匯給其他帳戶擁有者。」

回應：

Dana is the only person who owns an account that has sent more than \$499 to an account owned by a different person in the last 24 months.

(Specifically, her account sent exactly \$500, which is the maximum transfer amount in the database).

# 路徑搜尋

「Find the shortest path of transfers between Account ID 16 and Account ID 20.」(找出帳戶 ID 16 和帳戶 ID 20 之間最短的轉移路徑)。

回應：

The shortest path between Account 16 and Account 20 is 1 hop.

This means there is a direct transfer from Account 16 to Account 20.

步驟 5 · 約 0 分鐘

## 5. 恭喜！

恭喜您完成本程式碼研究室。

### 涵蓋內容

在本程式碼研究室中，您已學會如何使用 Spanner MCP 伺服器設定 Gemini CLI，並針對 Spanner Graph 資料發出自然語言查詢，以發掘洞察資料。以下是主要里程碑的摘要：

- **代管基礎架構**：啟用遠端 Spanner MCP 伺服器，不必使用本機 Proxy 或 MCP Toolbox。
  - **圖形整合**：設定具有節點和邊緣的 Spanner 屬性圖形 (FinGraph)，代表複雜的關係。
  - **原生工具**：使用標準 URI 配置，將 Gemini CLI 直接連線至受管理 Spanner 端點。
  - **自然語言 SQL 和 GQL**：使用圖形專屬提示來訓練伺服器，讓 LLM 執行多重跳躍式遍歷和週期偵測，不必手動編寫 SQL/GQL 程式碼。
  - **企業級技術**：探討受管理 MCP 如何提供可擴充的驗證路徑，供 AI 代理與營運資料互動。
-